

aneek4ever/Mipt\_android\_2020

Kotlin Koans: The Best Way To Learn Kotlin

play.kotlinalg.org/koans/Introduction/Hello,%20world!/Task.kt

Kotlin

Solutions Docs Community Teach Play

Progress: 2%

Introduction

✓ Hello, world!

Named arguments

Default arguments

Triple-quoted strings

String templates

Nullable types

Nothing type

Lambdas

Classes

Conventions

Collections

Properties

Builders

Generics

fun start(): String = "OK"

✓ Passed: test0k

×

Next koan

1/8

Simple Functions

Check out the [function syntax](#) and change the code to make the function `start` return the string `"OK"`.

In the Kotlin Koans tasks, the function `TODO()` will throw an exception. To complete Kotlin Koans, you need to replace this function invocation with meaningful code according to the problem.

Revert Show answer



Default and named arguments help to minimize the number of overloads and improve the readability of the function invocation. The library function `joinToString` is declared with default values for parameters:

It can be called on a collection of Strings.

Show answer

Next koan

3/8

## Default arguments

- Imagine you have several overloads of 'foo()' in Java:

```
public String foo(String name, int number, boolean toUpper) {
    return (toUpperCase ? name.toUpperCase() : name) + number;
}

public String foo(String name, int number) {
    return foo(name, number, false);
}

public String foo(String name, boolean toUpperCase) {
    return foo(name, 42, toUpperCase);
}

public String foo(String name) {
    return foo(name, 42);
}
```

You can replace all these Java overloads with one function in Kotlin. Change the declaration of the `foo` function in a way that makes the code using `foo` compile. Use [default and named arguments](#).

```
fun useFoo() = listOf(  
    foo("a"),  
    foo("b", number = 1),  
    foo("c", toUpperCase = true),  
    foo(name = "d", number = 2, toUpperCase = true)  
)
```

✔ Passed: testDefaultAndNamedParams

[Previous](#)

Next koan

Revert

Show answer

4/8

### Triple-quoted strings

- Learn about the [different string literals and string templates in Kotlin](#).

You can use the handy library functions `trimIndent` and `trimMargin` to format multiline triple-quoted strings in accordance with the surrounding code.

Replace the `trimIndent` call with the `trimMargin` call taking `#` as the prefix value so that the resulting string doesn't contain the prefix character.

Show answer

✔ Passed: testSolution

**Next koan**

5/8



The following pattern matches a date in the format 13.06.1992 (two digits, a dot, two digits, a dot, four digits):

Using the `month` variable, rewrite this pattern in such a way that it matches the date in the format `13 JUN 1992` (two digits, one whitespace, a month abbreviation, one whitespace, four digits).

Show answer

- ✓ Passed: match1
- ✓ Passed: match
- ✓ Passed: doNotMatch

Next koan

6/8

## Nullable types

- Learn about [null safety and safe calls](#) in Kotlin and rewrite the following Java code so that it only has one `if` expression:

```
public void sendMessageToClient(
    @Nullable Client client,
    @Nullable String message,
    @NotNull Mailer mailer
) {
    if (client == null || message == null) return;

    PersonalInfo personalInfo = client.getPersonalInfo();
    if (personalInfo == null) return;

    String email = personalInfo.getEmail();
    if (email == null) return;

    mailer.sendMessage(email, message);
}
```

Show answer

[Next koan](#)

[Previous](#)

718



Specify `Nothing` return type for the `failWithWrongAge` function. Note that without the `Nothing` type, the `checkAge` function doesn't compile because the compiler assumes the `age` can be `null`.

Show answer

- ✔ Passed: testNegative
- ✔ Passed: testLargeNumber

Next koan



Progress:19%

## ▼ Introduction

- ✓ Hello, world!
- ✓ Named arguments
- ✓ Default arguments
- ✓ Triple-quoted strings
- ✓ String templates
- ✓ Nullable types
- ✓ Nothing type
- ✓ Lambdas

## ► Classes

## ► Conventions

► Collections

### ► Properties

## ► Builders

### ► Generics

```
fun containsEven(collection: Collection<Int>): Boolean =  
    collection.any { it % 2 == 0 }
```

- ✔ Passed: contains
- ✔ Passed: notContain

## Previous

Next koar

84

## Lambdas

Kotlin supports functional programming. Learn about [lambdas](#) in Kotlin.

Pass a lambda to the `any` function to check if the collection contains an even number. The `any` function gets a predicate as an argument and returns true if at least one element satisfies the predicate.

Rever

Show answer





```
public class Person {
    private final String name;
    private final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}
```

Revert Show answer

- ```
data class Person(val name: String, val age: Int)

fun getPeople(): List<Person> {
    return listOf(Person("Alice", 29), Person("Bob", 31))
}

fun comparePeople(): Boolean {
    val p1 = Person("Alice", 29)
    val p2 = Person("Alice", 29)
    return p1 == p2 // should be true
}
```



Next koan

Progress: 23%

- ▶ Introduction
- ▼ Classes
  - ✓ Data classes
  - ✓ Smart casts
    - Sealed classes
    - Rename on import
    - Extension functions
- ▶ Conventions
- ▶ Collections
- ▶ Properties
- ▶ Builders
- ▶ Generics

```
fun eval(expr: Expr): Int =
    when (expr) {
        is Num -> expr.value
        is Sum -> eval(expr.left) + eval(expr.right)
        else -> throw IllegalArgumentException("Unknown expression")
    }

interface Expr
class Num(val value: Int) : Expr
class Sum(val left: Expr, val right: Expr) : Expr
```

- ✔ Passed: testNum
- ✔ Passed: testSum
- ✔ Passed: testRecursion

Next koan



## Smart casts

Rewrite the following Java code using **smart casts** and the **when** expression:

```
public int eval(Expr expr) {
    if (expr instanceof Num) {
        return ((Num) expr).getValue();
    }
    if (expr instanceof Sum) {
        Sum sum = (Sum) expr;
        return eval(sum.getLeft()) + eval(sum.getRight());
    }
    throw new IllegalArgumentException("Unknown expression");
}
```

Show answer

3/5



Revert Show answer

- ✔ Passed: testNum
- ✔ Passed: testSum
- ✔ Passed: testRecursion

Next koan

4/5



Uncomment the code and make it compile.  
Rename `Random` from the `kotlin` package to `KRandom`, and `Random` from the `java` package to `JRandom`.

Show answer

✔ Passed: testRandom

Next koan

5/5



`Pair` is a class defined in the standard library:

In the case of `Int`, the denominator is 1.

Show answer

- ✓ Passed: testPairExtension
- ✓ Passed: testIntExtension

Next koan

1/5



Note that when you override a member in Kotlin, the `override` modifier is mandatory.

Show answer

- ✔ Passed: testAfter
- ✔ Passed: testSame
- ✔ Passed: testBefore

Next koan

aneek4ever/Mpt\_android\_2020

Kotlin Koans: The Best Way To Learn Kotlin

play.kotlinalg.org/koans/Conventions/Ranges/Task.kt

Kotlin

Solutions

Docs

Community

Teach

Play

Progress: 35%

Introduction

Classes

Conventions

Comparison

Ranges

MyDate.kt

For loop

Operators overloading

Invoke

Collections

Properties

Builders

Generics

fun checkInRange(date: MyDate, first: MyDate, last: MyDate): Boolean {  
    return date in first..last  
}

2/5

## Ranges

Using `ranges` implement a function that checks whether the date is in the range between the first date and the last date (inclusive).

You can build a range of any comparable elements. In Kotlin `in` checks are translated to the corresponding `contains` calls and `..` to `rangeTo` calls:

```
val list = listOf("a", "b")  
"a" in list // list.contains("a")  
"a" !in list // !list.contains("a")  
  
date1..date2 // date1.rangeTo(date2)
```

Revert

Show answer

Previous

Next koan





Make the class `DateRange` implement `Iterable<MyDate>`, so that it can be iterated over. Use the function `MyDate.followingDate()` defined in `DateUtil.kt`; you don't have to implement the logic for finding the following date on your own.

Revert Show answer

- ✔ Passed: testIterateOverDateRange
- ✔ Passed: testIterateOverEmptyRange

- ▶ Introduction
- ▶ Classes
- ▼ Conventions
  - ✔ Comparison
  - ✔ Ranges
  - ✔ For loop
- DateUtil.kt
- MyDate.kt
- Operators overloading
- Invoke
- ▶ Collections
- ▶ Properties
- ▶ Builders
- ▶ Generics

4/5



First, add the extension function `plus()` to `MyDate`, taking the `TimeInterval` as an argument. Use the utility function `MyDate.addTimeIntervals()` declared in `DateUtil.kt`

Revert

Show answer

- ✔ Passed: testOneMonth
- ✔ Passed: testAddOneTimeInterval
- ✔ Passed: testMonthChange



5/5



You can add an `invoke` extension for any class, but it's better not to overuse it:

Implement the function `Invokable.invoke()` to count the number of times it is invoked.

Show answer

- ✔ Passed: testInvokeTwice
- ✔ Passed: testNumberOfInvocations

Next koan

Progress: 44%

- ▶ Introduction
- ▶ Classes
- ▶ Conventions
- ▼ Collections
  - ✓ Introduction
    - Shop.kt
    - Sort
    - Filter map
    - All Any and other predicates
    - Associate
    - GroupBy
    - Partition
    - FlatMap
    - Max min
    - Sum
    - Fold and reduce
    - Compound tasks
    - Sequences
    - Getting used to new style
- ▶ Properties
- ▶ Builders
- ▶ Generics

```
fun Shop.getSetOfCustomers(): Set<Customer> =  
    customers.toSet()
```

✓ Passed: testSetOfCustomers

1/14

## Introduction

This section was inspired by [GS Collections Kata](#).

Kotlin can be easily mixed with Java code. Default collections in Kotlin are all Java collections under the hood. Learn about [read-only and mutable views on Java collections](#).

The [Kotlin standard library](#) contains lots of extension functions that make working with collections more convenient. For example, operations that transform a collection into another one, starting with 'to': `toSet` or `toList`.

Implement the extension function `Shop.getSetOfCustomers()`. The class `Shop` and all related classes can be found in `Shop.kt`.

Revert

Hide answer

2/14



implement a function for returning the list of customers, sorted in descending order by the number of orders they have made. Use `sortedDescending` or `sortedByDescending`.

```
val strings = listOf("bbb", "a", "cc")
strings.sorted() ==
    listOf("a", "bbb", "cc")

strings.sortedBy { it.length } ==
    listOf("a", "cc", "bbb")

strings.sortedDescending() ==
    listOf("cc", "bbb", "a")

strings.sortedByDescending { it.length } ==
    listOf("bbb", "cc", "a")
```

Show answer

✔ Passed: testGetCustomersSortedByNumberOfOrders

Next koan

aneK4ever/Mipt\_android\_202

Kotlin Koans: The Best Way To

play.kotlinlang.org/koans/Collections/Filter%20map/Task.kt

Kotlin

Solutions

Docs

Community

Teach

Play

Progress: 49%

Introduction

Classes

Conventions

Collections

- Introduction
- Sort
- Filter map
  - Shop.kt

All Any and other predicates

Associate

GroupBy

Partition

FlatMap

Max min

Sum

Fold and reduce

Compound tasks

Sequences

Getting used to new style

Properties

Builders

// Find all the different cities the customers are from  
**fun** Shop.getCustomerCities(): Set<City> =  
    customers.map { it.city }.toSet()  
  
// Find the customers living in a given city  
**fun** Shop.getCustomersFrom(city: City): List<Customer> =  
    customers.filter { it.city == city }

Passed: testCitiesCustomersAreFrom

Passed: testCustomersFromCity

Previous

Next koan

3/14

## Filter; map

Learn about [mapping](#) and [filtering](#) a collection.

Implement the following extension functions using the `map` and `filter` functions:

- Find all the different cities the customers are from
- Find the customers living in a given city

```
val numbers = listOf(1, -1, 2)
numbers.filter { it > 0 } == listOf(1, 2)
numbers.map { it * it } == listOf(1, 1, 4)
```

Revert

Show answer



4/14



Implement the following functions using `all`, `any`, `count`, `find`:

- ```
val numbers = listOf(-1, 0, 2)
val isZero: (Int) -> Boolean = { it == 0 }
numbers.any(isZero) == true
numbers.all(isZero) == false
numbers.count(isZero) == 1
numbers.find { it > 0 } == 2
```

Show answer

```
// Return true if all customers are from a given city
fun Shop.checkAllCustomersAreFrom(city: City): Boolean =
    customers.all { it.city == city }

// Return true if there is at least one customer from a given city
fun Shop.hasCustomerFrom(city: City): Boolean =
    customers.any { it.city == city }

// Return the number of customers from a given city
fun Shop.countCustomersFrom(city: City): Int =
    customers.count { it.city == city }

// Return a customer who lives in a given city, or null if there is none
fun Shop.findCustomerFrom(city: City): Customer? =
    customers.find { it.city == city }
```

- ✔ Passed: testAnyCustomerFromCity
- ✔ Passed: testAnyCustomerIsFromCity
- ✔ Passed: testCountCustomersFromCity
- ✔ Passed: testAllCustomersAreFromCity

Next koan





aneK4ever/Mipt\_android\_202

Kotlin Koans: The Best Way To

+

← → ↺ 🏠 🌐 play.kotlinalg.org/koans/Collections/GroupBy/Task.kt 🔄 📄 ☆ 📁 👤 ⋮

Kotlin

Solutions Docs Community Teach Play 🔍

Progress: 56%

▶ Introduction

▶ Classes

▶ Conventions

▼ Collections

- ✓ Introduction
- ✓ Sort
- ✓ Filter map
- ✓ All Any and other predicates
- ✓ Associate
- ✓ GroupBy
  - Shop.kt
- Partition
- FlatMap
- Max min
- Sum
- Fold and reduce
- Compound tasks
- Sequences
- Getting used to new style

▶ Properties

▶ Builders

// Build a map that stores the customers living in a given city

**fun** Shop.groupCustomersByCity(): Map<City, List<Customer>> = customers.groupBy { it.city }

✓ Passed: testGroupCustomersByCity

✕

Previous

Next koan

6/14

## Group By

Learn about [grouping](#). Use `groupBy` to implement the function to build a map that stores the customers living in a given city.

```
val result = listOf("a", "b", "ba", "ccc", "ad").groupBy { it.length }

result == mapOf(
    1 to listOf("a", "b"),
    2 to listOf("ba", "ad"),
    3 to listOf("ccc")
)
```

Revert Show answer

```
// Return a customer who has placed the maximum amount of orders
fun Shop.getCustomerWithMaxOrders(): Customer? =
    customers.maxByOrNull { it.orders.size }

// Return the most expensive product that has been ordered by the given customer
fun Shop.getMostExpensiveProductBy(customer: Customer): Product? =
    customer.orders
        .flatMap(Order::products)
        .maxByOrNull(Product::price)
```

- ✔ Passed: testTheMostExpensiveOrderedProduct
- ✔ Passed: testCustomerWithMaximumNumberOfOrders

Next koan

### Max min

Learn about [collection aggregate operations](#).

Implement two functions:

- The first should return the customer who has placed the most amount of orders in this shop
- The second should return the most expensive product that the given customer has ordered

The functions `maxOrNull`, `minOrNull`, `maxByOrNull`, and `minByOrNull` might be helpful.

```
listOf(1, 42, 4).maxOrNull() == 42
listOf("a", "ab").minByOrNull{String::length} == "a"
```

You can use [callable references](#) instead of lambdas. It can be especially helpful in call chains, where it occurs in different lambdas and has different types. Implement the `getMostExpensiveProductBy` function using callable references.

Show answer

- ✓ Introduction
- ✓ Sort
- ✓ Filter map
- ✓ All Any and other predicates
- ✓ Associate
- ✓ GroupBy
- ✓ Partition
- Shopt.kt
- ✓ FlatMap
- ✓ Max min
- Sum
- Fold and reduce
- Compound tasks
- Sequences
- Getting used to new style

Next koan

## Partition

Then implement a function for returning customers who have more undelivered orders than delivered orders using `partition`.

```
positive == listOf(1, 3, 2)
negative == listOf(-4, -11)
```

Show answer

Revert

Show answer

[Previous](#)

Next koan

aneek4ever/Mipt\_android\_202

Kotlin Koans: The Best Way To

play.kotlinlang.org/koans/Collections/Sum/Task.kt

Kotlin

Solutions

Docs

Community

Teach

Play

Progress: 65%

► Introduction

► Classes

► Conventions

▼ Collections

✓ Introduction

✓ Sort

✓ Filter map

✓ All Any and other predicates

✓ Associate

✓ GroupBy

✓ Partition

✓ FlatMap

✓ Max min

✓ Sum

Shop.kt

Fold and reduce

Compound tasks

Sequences

Getting used to new style

► Properties

► Builders

► Generics

// Return the sum of prices for all the products ordered by a given customer

**fun** moneySpentBy(customer: Customer): Double =  
    customer.orders.flatMap { it.products }.sumOf { it.price }

✓ Passed: testGetTotalOrderPrice

✓ Passed: testGetTotalOrderPrice1

Previous

Next koan

10/14

►

## Sum

Implement a function that calculates the total amount of money the customer has spent: the sum of the prices for all the products ordered by a given customer. Note that each product should be counted as many times as it was ordered.

Use `sum` on a collection of numbers or `sumOf` to convert the elements to numbers first and then sum them up.

```
listOf(1, 5, 3).sum() == 9  
listOf("a", "b", "cc").sumOf { it.length } == 4
```

Revert

Show answer

✔ Passed: testGetProductsOrderedByAllCustomers

Next koan

## Fold and reduce

You can use the `Customer.getOrderedProducts()` defined in the previous task (copy its implementation).

You might also need the [intersect](#) function.

Show answer



```
// Find the most expensive product among all the delivered products
// ordered by the customer. Use "Order.isDelivered" flag.
fun findMostExpensiveProductBy(customer: Customer): Product? {
    return customer
        .orders
        .filter { Order::isDelivered }
        .flatMap { Order::products }
        .maxByOrNull { Product::price }
}

// Count the amount of times a product was ordered.
// Note that a customer may order the same product several times.
fun Shop.getNumberOfTimesProductWasOrdered(product: Product): Int {
    return customers
        .flatMap { Customer::getOrderedProducts }
        .count { it == product }
}

fun Customer.getOrderedProducts(): List<Product> =
    orders.flatMap { Order::products }
```

- ✔ Passed: testNumberOfTimesEachProductWasOrdered
- ✔ Passed: testMostExpensiveDeliveredProduct

Next koan

9

9

9

- 9

9

9

Show answer

- Properties
- Builders
- Generics

- ✔ Passed: testNumberOfTimesEachProductWasOrdered
- ✔ Passed: testMostExpensiveDeliveredProduct

Next koan

Copy the implementation from the previous task and modify it in a way that the operations on sequences are used.

Show answer



✔ Passed: testPropertyWithCounter

[Next koan](#)

Learn about [properties](#) in Kotlin.

Revert

Show answer

aneek4ever/Mipt\_android\_2020

Kotlin Koans: The Best Way To

play.kotlinlang.org/koans/Properties/Lazy%20property/Task.kt

Kotlin

Solutions Docs Community Teach Play

Progress: 79%

Introduction

Classes

Conventions

Collections

Properties

Properties

Lazy property

Delegates examples

Delegates how it works

Builders

Generics

```
class LazyProperty(val initializer: () -> Int) {  
    var value: Int? = null  
    val lazy: Int  
        get() {  
            if (value == null) {  
                value = initializer()  
            }  
            return value!!  
        }  
}
```

Passed: testLazy

Passed: initializedOnce

Previous

Next koan

2/4

Lazy property

Add a custom getter to make the val lazy really lazy. It should be initialized by invoking initializer() during the first access.

You can add any additional properties as you need.

Do not use delegated properties!

Revert

Show answer

aneek4ever/Mipt\_android\_2020

Kotlin Koans: The Best Way To

play.kotlinlang.org/koans/Properties/Delegates%20examples/Task.kt

Solutions

Docs

Community

Teach

Play

Kotlin

Progress: 81%

Introduction

Classes

Conventions

Collections

Properties

Properties

Lazy property

Delegates examples

Delegates how it works

Builders

Generics

```
class LazyProperty(val initializer: () -> Int) {  
    val lazyValue: Int by lazy(initializer)  
}
```

Passed: testLazy

Passed: initializedOnce

Previous

Next koan

3/4

Delegates example

Learn about [delegated properties](#) and make the property lazy using delegates.

Revert

Show answer

✔ Passed: testDate

[Previous](#)

Next koan

## Delegates

You can declare your own `delegates`. Implement the methods of the class `EffectiveDate` so you can delegate to it. Store only the time in milliseconds in the `timeInMillis` property.

Use the extension functions `MyDate.toMillis()` and `Long.toDate()`, defined in `MyDate.kt`.

Revert

Show answer



- Introduction
- Classes
- Conventions
- Collections
- Properties
- ▼ Builders
  - ✔ [Function literals with receiver](#)
    - String and map builders
    - The function apply
    - Html builders
    - Builders how it works
    - Builders implementation
- Generics

✔ Passed: testIsOddAndIsEven

Next koan

You can declare `isEven` and `isOdd` as values that can be called as extension functions. Complete the declarations in the code.

Show answer

ane4ever/Mipt\_android\_202

Kotlin Koans: The Best Way To

play.kotlinlang.org/koans/Builders/String%20and%20map%20builders/Task.kt

Kotlin

Solutions

Docs

Community

Teach

Play

Progress: 88%

► Introduction

► Classes

► Conventions

► Collections

► Properties

▼ Builders

✓ Function literals with receiver

✓ String and map builders

The function apply

Html builders

Builders how it works

Builders implementation

► Generics

import java.util.HashMap

fun <K, V> buildMutableMap(build: HashMap<K, V>().() -> Unit): Map<K, V> {  
    val map = HashMap<K, V>()  
    map.build()  
    return map  
}

fun usage(): Map<Int, String> {  
    return buildMutableMap {  
        put(0, "0")  
        for (i in 1..10) {  
            put(i, "\$i")  
        }  
    }  
}

✓ Passed: testBuildMap

Previous

Next koan

2/6

String and map builders

Function literals with receiver are very useful for creating builders, for example:

fun buildString(build: StringBuilder.() -> Unit): String {  
    val stringBuilder = StringBuilder()  
    stringBuilder.build()  
    return stringBuilder.toString()  
}  
  
val s = buildString {  
    this.append("Numbers: ")  
    for (i in 1..3) {  
        // 'this' can be omitted  
        append(i)  
    }  
}  
  
s == "Numbers: 123"

Implement the function `buildMutableMap` that takes a parameter (of extension function type), creates a new `HashMap`, builds it, and returns it as a result. Note that starting from 1.3.70, the standard library has a similar `buildMap` function.

Revert

Show answer

- ```
fun <T> T.myApply(f: T.() -> Unit): T {
    f()
    return this
}

fun createString(): String {
    return StringBuilder().myApply {
        append("Numbers: ")
        for (i in 1..10) {
            append(i)
        }
    }.toString()
}

fun createMap(): Map<Int, String> {
    return hashMapOf<Int, String>().myApply {
        put(0, "0")
        for (i in 1..10) {
            put(i, "$i")
        }
    }
}
```

Next koan



Learn about the other [scope functions](#) and how to use them.

Show answer

4/8

Show answer

- ▶ Introduction
- ▶ Classes
- ▶ Conventions
- ▶ Collections
- ▶ Properties
- ▼ Builders
  - ✔ Function literals with receiver
  - ✔ String and map builders
  - ✔ The function apply
  - ✔ Html builders
  - ✔ [Builders how it works](#)
  - Builders implementation
- ▶ Generics

Answer the questions below

```
tr {
  td {
    text("Product")
  }
  td {
    text("Popularity")
  }
}
```

- a. a special built-in syntactic construct
- b. a function declaration
- c. a function invocation

```
tr (color = "yellow") {
  td {
    text("Product")
  }
  td {
    text("Popularity")
  }
}
```

color is:

- ▶ Introduction
- ▶ Classes
- ▶ Conventions
- ▶ Collections
- ▶ Properties
- ▼ Builders
  - ✓ Function literals with receiver
  - ✓ String and map builders
  - ✓ The function apply
  - ✓ Html builders
  - ✓ Builders how it works
  - ✓ [Builders implementation](#)
- ▶ Generics

2000

## Builders implementation

Learn more about [type-safe builders](#).

Show answer

- ✔ Passed: testPartitionLettersAndOtherSymbols
- ✔ Passed: testPartitionWordsAndLines

[Previous](#)

Learn about [generic functions](#). Make the code compile by implementing a `partitionTo` function that splits a collection into two collections according to the predicate.

There is a `partition()` function in the standard library that always returns two newly created lists. Write a function that splits the collection into two collections given as arguments. The signature of the `toCollection()` function from the standard library might help you.

Revert

Show answer